Docker Volumes and Docker Compose: Cars CRUD Workshop

Goal: Understand persistent container storage, then use Docker Compose to run the existing **FastAPI + plain HTML** Cars CRUD application with a separate **MySQL** database container.

This is a follow-up to `DOCKER_GUIDE.md`. You should already have Docker installed and have built the Cars CRUD Docker image once. Run commands from the project root (`cars-crud/`).

> **Two stages, two databases:** Part 1 uses the original **SQLite** application in one container. Part 2 changes the running application to **MySQL** in two containers. SQLite data does **not** automatically transfer into MySQL. For training, you can start with a fresh MySQL database.

Part 1 - Docker volumes

1. Why volumes exist

A container has a writable filesystem layer, but **that layer is deleted when the container is removed**. Rebuilding an image or recreating a container should not erase important data such as car records, user uploads, or database files.

A **Docker volume** stores data separately from the container. Docker manages its location and lifecycle. A new container can mount the same volume and see the files created by the previous container.

```
Without a volume:
  container -> /data/cars.db -> remove container -> database lost

With a named volume:
  container A -> /data/cars.db -> [cars-data volume]
  remove A                              |
  container B -> /data/cars.db --------+   same saved data
```

Storage type	What it is	Typical use
Named volume	Docker-managed persistent directory	SQLite/MySQL data, uploads
Bind mount	Specific folder from your computer mapped into a container	Editing source files locally
Anonymous volume	Docker-managed volume without your chosen name	Short-lived or image-defined storage
tmpfs mount	Files held in memory (Linux containers)	Temporary sensitive or disposable data

Volumes are not backups. Keep a separate backup if losing the data would matter. Do not mount the same live SQLite file as a writeable database in multiple application containers.

2. Give our SQLite container a named volume

The Dockerfile from `DOCKER_GUIDE.md` already contains:

```
RUN mkdir -p /data && chown 10001:10001 /data
USER 10001:10001
ENV CARS_DB_PATH=/data/cars.db
```

This makes the application store SQLite data at `/data/cars.db`, inside the container. We will mount a volume at `/data`.

First build the app if needed:

```
docker build -t cars-crud:local .
```

For a local machine where Docker Hub is slow, use the course's Azure proxy as described in the earlier guide:

```
docker build --build-arg
BASE_IMAGE=efreiwut.azurecr.io/cache/library/python:3.12-slim -t cars-
crud:local .
```

Create a **named volume** (Docker would also create it automatically when first used):

```
docker volume create cars-data
```

If your earlier `cars-app` container already exists, stop and remove that container before reusing its name. Do **not** delete the `cars-data` volume.

```
docker stop cars-app
docker rm cars-app
```

The two commands above apply **only if** `cars-app` already exists; otherwise skip them.

Run the application:

```
docker run -d --name cars-app \
  -p 127.0.0.1:8000:8000 \
```

```
  --mount type=volume,source=cars-data,target=/data \
  cars-crud:local
```

Open http://localhost:8000/ and create a car. You can also use http://localhost:8000/docs.

The important part is:

```
source=cars-data    -> the Docker volume holding the data
target=/data        -> where the volume appears inside the container
```

Windows note: The multi-line command above uses backslashes for macOS/Linux shells. On PowerShell, put the full `docker run` command on one line or use PowerShell's backtick continuation character.

3. Prove the car survives container replacement

```
docker stop cars-app
docker rm cars-app

docker run -d --name cars-app \
  -p 127.0.0.1:8000:8000 \
  --mount type=volume,source=cars-data,target=/data \
  cars-crud:local
```

Refresh the page. The car you created should still be there, because it lives in `cars-data`, not in the removed container.

4. Commands to remember

Command	Purpose
`docker volume create NAME`	Create a named volume
`docker volume ls`	List volumes
`docker volume inspect NAME`	Inspect its configuration and mount location
`docker inspect cars-app`	See the container's `Mounts` section
`docker exec -it cars-app sh`	Open a shell in the running container
`docker volume rm NAME`	Delete a volume **after** containers using it are removed
`docker volume prune`	Delete unused local volumes; potentially destructive

Command	Purpose
`docker run --mount type=volume,source=NAME,target=/data ...`	Mount a named volume
`docker run -v NAME:/data ...`	Shorter equivalent for a named volume

Useful checks:

```
docker volume ls
docker volume inspect cars-data
docker inspect cars-app
```

Be careful: `docker volume rm cars-data` deletes the saved SQLite database. `docker rm cars-app` by itself does **not** remove a named volume. Always verify which volume contains your data before deleting it.

Bind mount comparison: `--mount type=bind,source=/absolute/host/path,target=/data` uses a folder you choose on your computer. It is useful for source code, but it is less portable and can cause permission issues. The Cars app runs as user `10001`, so **named volumes are simpler for this exercise**.

Part 2 - Docker Compose and MySQL

5. What Docker Compose does

Docker Compose describes multiple services in one YAML file, normally named `compose.yaml`. It can **build images**, **pull existing images**, **create containers**, **connect them on a network**, **mount volumes**, and **start or stop the whole application** with one command.

For our updated app:

```
Browser
  |
  v
localhost:8000
  |
  v
[app service: FastAPI + raw HTML]
  |
  | MySQL connection to host "db", port 3306
  v
[db service: MySQL 8.4]
  |
```

```
  v
[mysql-data named volume: /var/lib/mysql]
```

The frontend stays plain HTML, CSS and JavaScript served by FastAPI. **Only the database storage implementation changes.**

Main Compose concepts

YAML key	Meaning in our example
`services`	Containers we want: `app` and `db`
`build`	Build `app` from our local Dockerfile
`build.args`	Pass `BASE_IMAGE` to the Dockerfile
`image`	Pull an existing image, such as `mysql:8.4`
`ports`	Expose FastAPI on `127.0.0.1:8000` on your computer
`environment`	Give containers database settings
`depends_on`	Wait for the database's health check before starting the app
`healthcheck`	Verify MySQL can accept a real query
`volumes`	Preserve MySQL files across container replacement

Compose automatically creates a private project network. The application reaches MySQL using the **service name `db`**, not `localhost`. Inside a container, `localhost` means **that same container**.

Build versus pull: `docker compose build app` builds our FastAPI image using its Dockerfile; the `db` service normally **pulls** the ready-made `mysql:8.4` image. You do not need to write a MySQL Dockerfile.

6. Prepare the original Python app for MySQL

Our starting app uses `app/database.py`, which calls Python's built-in `sqlite3` module. **Changing only `compose.yaml` is not enough:** SQLite code cannot talk to MySQL. We will keep the SQLite backend for the existing tests and add a small MySQL backend for Compose.

The following changes are compatible with the earlier unit, integration and API tests. Those existing tests will **continue using temporary SQLite databases**; they do not automatically test the new MySQL backend.

Step A - Add a Python MySQL driver

Open `requirements.txt` and add this line after the existing dependencies:

```
PyMySQL[rsa]>=1.1,<2
```

`PyMySQL` sends parameterized queries to MySQL; the `rsa` extra supports authentication methods used by modern MySQL servers. `requirements-dev.txt` already includes `requirements.txt`, so no change is needed there.

Step B - Create `app/mysql_database.py`

Copy this complete file. It offers the same six CRUD functions as the SQLite module. The `_database_path` argument is intentionally unused by MySQL; it keeps the existing FastAPI route calls the same in both modes.

```
"""MySQL CRUD adapter matching the existing SQLite module's public
functions.

The Path argument is retained so the FastAPI routes and SQLite test
fixtures
can use either backend without changing their function calls.
"""

import os
from pathlib import Path

from app.schemas import Car, CarInput


def connect():
    # Import only when the MySQL backend is selected. SQLite-only tests
stay isolated.
    import pymysql
    from pymysql.constants import CLIENT
    from pymysql.cursors import DictCursor

    return pymysql.connect(
        host=os.environ.get("MYSQL_HOST", "db"),
        port=int(os.environ.get("MYSQL_PORT", "3306")),
        user=os.environ["MYSQL_USER"],
        password=os.environ["MYSQL_PASSWORD"],
        database=os.environ["MYSQL_DATABASE"],
        charset="utf8mb4",
        cursorclass=DictCursor,
        autocommit=True,
        client_flag=CLIENT.FOUND_ROWS,
        connect_timeout=10,
    )


def initialize(_database_path: Path) -> None:
    with connect() as connection, connection.cursor() as cursor:
        cursor.execute(
            """
            CREATE TABLE IF NOT EXISTS cars (
                id INT NOT NULL AUTO_INCREMENT PRIMARY KEY,
                make VARCHAR(60) NOT NULL,
```

```
                model VARCHAR(60) NOT NULL,
                year INT NOT NULL,
                color VARCHAR(40) NOT NULL DEFAULT '',
                CONSTRAINT valid_make CHECK (CHAR_LENGTH(TRIM(make)) > 0),
                CONSTRAINT valid_model CHECK (CHAR_LENGTH(TRIM(model)) >
0),
                CONSTRAINT valid_year CHECK (year BETWEEN 1886 AND 2100)
            ) CHARACTER SET utf8mb4
            """
        )


def list_cars(_database_path: Path) -> list[Car]:
    with connect() as connection, connection.cursor() as cursor:
        cursor.execute("SELECT id, make, model, year, color FROM cars ORDER
BY id DESC")
        return [Car(**row) for row in cursor.fetchall()]


def get_car(_database_path: Path, car_id: int) -> Car | None:
    with connect() as connection, connection.cursor() as cursor:
        cursor.execute(
            "SELECT id, make, model, year, color FROM cars WHERE id = %s",
(car_id,)
        )
        row = cursor.fetchone()
        return Car(**row) if row else None


def create_car(_database_path: Path, car: CarInput) -> Car:
    with connect() as connection, connection.cursor() as cursor:
        cursor.execute(
            "INSERT INTO cars (make, model, year, color) VALUES (%s, %s,
%s, %s)",
            (car.make, car.model, car.year, car.color),
        )
        return Car(id=cursor.lastrowid, **car.model_dump())


def update_car(_database_path: Path, car_id: int, car: CarInput) -> Car |
None:
    with connect() as connection, connection.cursor() as cursor:
        matched = cursor.execute(
            """UPDATE cars
               SET make = %s, model = %s, year = %s, color = %s
               WHERE id = %s""",
            (car.make, car.model, car.year, car.color, car_id),
        )
        return Car(id=car_id, **car.model_dump()) if matched else None


def delete_car(_database_path: Path, car_id: int) -> bool:
    with connect() as connection, connection.cursor() as cursor:
        deleted = cursor.execute("DELETE FROM cars WHERE id = %s",
```

```
    (car_id,))
            return deleted == 1
```

Important SQL differences: SQLite uses `?` for query parameters, while **PyMySQL uses `%s`**. Both are bound parameters, **not string formatting**. MySQL uses `AUTO_INCREMENT` instead of SQLite's `AUTOINCREMENT`. The adapter opens and closes a connection for each operation; this is sufficient for the training exercise, not a high-traffic production pattern.

Step C - Select a backend in `app/main.py`

1. Change the import from:

```
from app import database
```

to:

```
from app import database, mysql_database
```

2. Inside `create_app()`, **immediately after** the code that assigns `db_path` and ends with `).resolve()`, add:

```
    # Tests passing a SQLite file path always keep using SQLite.
    storage = (
        mysql_database
        if database_path is None and os.getenv("CARS_DB_BACKEND") ==
"mysql"
        else database
    )
```

3. In `create_app()`, replace these function calls (only these calls; do not rename the import itself):

Before	After
`database.initialize(db_path)`	`storage.initialize(db_path)`
`database.list_cars(db_path)`	`storage.list_cars(db_path)`
`database.get_car(db_path, car_id)`	`storage.get_car(db_path, car_id)`
`database.create_car(db_path, car)`	`storage.create_car(db_path, car)`
`database.update_car(db_path, car_id, car)`	`storage.update_car(db_path, car_id, car)`
`database.delete_car(db_path, car_id)`	`storage.delete_car(db_path, car_id)`

No HTML, JavaScript, schemas, or API paths need to change. Without `CARS_DB_BACKEND=mysql`, the application still uses SQLite. Explicit `create_app(database_path)` calls from existing tests always use SQLite, regardless of the environment variable.

Step D - Keep the Dockerfile

Use the **same Dockerfile from the previous guide** (`ARG BASE_IMAGE=python:3.12-slim`, `COPY requirements.txt`, `COPY app/`, Uvicorn command, etc.). The updated `requirements.txt` will install the MySQL driver when Compose rebuilds the image.

No additional Dockerfile is needed for MySQL; Compose uses the official MySQL image for the database service. The existing SQLite `ENV CARS_DB_PATH=/data/cars.db` is harmless in MySQL mode because the MySQL adapter does not use that path.

7. Create the Compose file

Create **`compose.yaml`** at the root of `cars-crud/`:

```
services:
  app:
    build:
      context: .
      args:
        BASE_IMAGE: ${BASE_IMAGE:-python:3.12-slim}
    ports:
      - "127.0.0.1:8000:8000"
    environment:
      CARS_DB_BACKEND: mysql
      MYSQL_HOST: db
      MYSQL_PORT: "3306"
      MYSQL_DATABASE: ${MYSQL_DATABASE:?Set MYSQL_DATABASE in .env}
      MYSQL_USER: ${MYSQL_USER:?Set MYSQL_USER in .env}
      MYSQL_PASSWORD: ${MYSQL_PASSWORD:?Set MYSQL_PASSWORD in .env}
    depends_on:
      db:
        condition: service_healthy

  db:
    image: ${MYSQL_IMAGE:-mysql:8.4}
    environment:
      MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD:?Set MYSQL_ROOT_PASSWORD
in .env}
      MYSQL_DATABASE: ${MYSQL_DATABASE:?Set MYSQL_DATABASE in .env}
      MYSQL_USER: ${MYSQL_USER:?Set MYSQL_USER in .env}
      MYSQL_PASSWORD: ${MYSQL_PASSWORD:?Set MYSQL_PASSWORD in .env}
    volumes:
      - mysql-data:/var/lib/mysql
    healthcheck:
      test: ["CMD-SHELL", "mysql -h 127.0.0.1 -u\"$$MYSQL_USER\" -
p\"$$MYSQL_PASSWORD\" \"$$MYSQL_DATABASE\" -e 'SELECT 1' >/dev/null 2>&1"]
      interval: 5s
      timeout: 5s
```

```
      retries: 20
      start_period: 30s

volumes:
  mysql-data:
```

How this file works

- `app.build.context: .` builds our code from the Dockerfile in the current directory.
- `app.ports` lets your computer reach FastAPI at http://localhost:8000. The `127.0.0.1` binding keeps this unauthenticated training app local.
- `MYSQL_HOST: db` resolves to the MySQL service on the Compose network.
- `db.image` selects the official MySQL image, rather than building it.
- `db.volumes` stores MySQL data at its normal data directory, `/var/lib/mysql`.
- `depends_on.condition: service_healthy` waits for a MySQL health check. A database container may be running before the database is actually ready.
- `volumes.mysql-data` declares a named volume for Compose to manage.
- The double dollar signs `$$MYSQL_USER` etc. in the health check tell Compose to leave those variables for the **database container** to expand at runtime.
- We deliberately **do not publish MySQL's port 3306** to the host. The app connects over the internal Compose network.

8. Create the local environment file

Create a file named `.env`, next to `compose.yaml`:

```
MYSQL_DATABASE=cars
MYSQL_USER=cars_user
MYSQL_PASSWORD=change_this_training_password
MYSQL_ROOT_PASSWORD=change_this_root_password
```

If you are using the companion reference project, you can instead copy `.env.example` to `.env` and then edit the values. Replace the example passwords with your own for local use. The original project's `.gitignore` **already excludes** `.env`; verify that this remains true. The `.dockerignore` from `DOCKER_GUIDE.md` also excludes it.

The `${NAME:?...}` references in Compose require variables from `.env` or your shell. `.env` is a convenient **local configuration file, not a production secrets store**. Do not commit it or post its contents in screenshots. `docker compose config` can print resolved credentials; use `docker compose config --quiet` to validate without displaying them.

Optional: faster image pulls in China

If Docker Hub is slow, the course's Azure registry proxy can be used **where the requested image and tag are available through it**. Add to `.env`:

```
BASE_IMAGE=efreiwut.azurecr.io/cache/library/python:3.12-slim
# Only if the proxy also serves this MySQL image:
MYSQL_IMAGE=efreiwut.azurecr.io/cache/library/mysql:8.4
```

The default values without overrides are `python:3.12-slim` for the Dockerfile and `mysql:8.4` for Compose. The course proxy is an **alternate source for pulling images**, not a destination for saving our database or application image. You can remove either override if the requested image is not available through the proxy.

9. Build and start both services

Make sure the old standalone SQLite container is stopped if it still uses port 8000:

```
docker stop cars-app
```

Skip this if `cars-app` is not running.

Check the Compose installation and YAML configuration:

```
docker compose version
docker compose config --quiet
```

Build the FastAPI image:

```
docker compose build app
```

Start the two services (the `--build` option also builds the app image when needed):

```
docker compose up --build -d
```

Docker Compose will:

1. Build the `app` image using the Dockerfile and updated Python dependencies.
2. Pull the `db` image if it is not available locally.
3. Create a shared network and the `mysql-data` volume.
4. Start MySQL and wait for its health check.
5. Start FastAPI, which creates the `cars` table if necessary.

Visit http://localhost:8000/. Create, view, edit, and delete a car. Use http://localhost:8000/docs to try the REST API directly.

Check the running services and logs:

```
docker compose ps
docker compose logs -f app
docker compose logs -f db
```

Press **Ctrl+C** to stop watching the logs. This does **not** stop the containers.

To see the records directly in MySQL (enter `MYSQL_PASSWORD` when prompted):

```
docker compose exec db mysql -u cars_user -p cars
```

Then run `SELECT * FROM cars;` and `exit` in the MySQL prompt. If you changed the `MYSQL_USER` or `MYSQL_DATABASE` values in `.env`, use those values instead.

10. Prove MySQL persistence

With at least one car saved through the website:

```
docker compose down
docker compose up -d
docker compose ps
```

Refresh http://localhost:8000/. Your car should still exist because the named **MySQL volume** survived `docker compose down`.

Danger: the following is intentionally destructive and not part of the persistence check:

```
docker compose down -v
```

`-v` removes the declared named volume along with the containers. For this tutorial, that means **deleting the MySQL database contents**. Only use it when you explicitly want to reset all exercise data. Changing MySQL initialization passwords or database names in `.env` will not reinitialize an already populated MySQL data volume.

11. Key Compose commands

Command	Purpose
`docker compose config --quiet`	Validate Compose YAML and required variables
`docker compose build app`	Build/rebuild only the FastAPI image
`docker compose build --no-cache app`	Rebuild FastAPI without cached image layers
`docker compose pull db`	Pull the selected MySQL image

Command	Purpose
`docker compose up -d`	Start the services in the background
`docker compose up --build -d`	Build needed images, then start services
`docker compose ps`	Show service/container status
`docker compose images`	Show images used by the services
`docker compose logs -f app`	Follow FastAPI logs
`docker compose logs -f db`	Follow MySQL logs
`docker compose exec app sh`	Open a shell in the FastAPI container
`docker compose exec db mysql -u cars_user -p cars`	Connect to MySQL inside its container
`docker compose stop`	Stop services without removing containers
`docker compose down`	Remove Compose containers and network, retain named volume
`docker compose down -v`	**Remove containers AND named volumes** (data loss)

`build:` vs `image:`: `build` creates an image from your code and Dockerfile. `image` references an image by name, typically for pulling. If you want Compose to give your built app a specific tag, you can add `image: cars-crud:mysql` under the `app` service. That name is local unless you tag and push it to a registry yourself.

12. Troubleshooting

Problem	Likely cause / fix
Port 8000 is already allocated	Stop the old `cars-app` or local Uvicorn server; or change host port to `8001:8000`.
`ModuleNotFoundError: pymysql`	Add `PyMySQL[rsa]>=1.1,<2` to `requirements.txt` and run `docker compose up --build -d`.
App cannot connect to MySQL	Check `docker compose ps`, `docker compose logs db`, credentials, and `MYSQL_HOST: db`.
MySQL never becomes healthy	Look at `docker compose logs db` and its initialization/errors; give first startup time.
New `.env` password does not work	Existing MySQL volumes retain their original users/passwords; environment variables initialize only a new data directory.
Cars from SQLite do not appear in MySQL	These are **separate databases**. Export/import is a separate migration task.

Problem	Likely cause / fix
`Permission denied` on SQLite file	Mount a named volume at `/data` and use the existing Dockerfile's `/data` ownership setup.
Unable to pull `mysql:8.4`	Verify Docker Hub access; use the Azure proxy only if that image/tag is supported.
MySQL accessible externally	Do **not** add a host `ports` mapping for `db` unless explicitly needed for local work.

13. Quick recap

- A **volume** lets state survive container replacement. SQLite uses `cars-data:/data`; MySQL uses `mysql-data:/var/lib/mysql`.
- A **Dockerfile** defines how to build **one image**.
- A **Compose file** describes how to run **multiple cooperating services** from built or pre-existing images.
- Our HTML/frontend and FastAPI API remain in one `app` service; MySQL runs in a separate `db` service.
- `docker compose up --build -d` builds/starts the system; `docker compose down` stops/removes it **without deleting named volumes**.

Official references

- Docker volumes
- Docker Compose Quickstart
- Compose build specification
- Startup order and health checks
- Compose variables and .env
- MySQL Docker Official Image